

1 Editorial: cartonase

1.1 Prezentarea problemei

Se dau n cartonase, fiecare cu un număr întreg. În funcție de valoarea c , trebuie să rezolvăm una dintre cele trei subpuncte:

- $c = 1$: Pentru o poziție dată poz , să se determine câte cartonase de la început până la poziția poz au valoarea cel puțin egală cu $v[poz]$.
- $c = 2$: Să se găsească toate pozițiile i pentru care maximul dintre primele i elemente este egal cu i .
- $c = 3$: Să se găsească toate pozițiile i pentru care cel mai mare element dintre primele i este strict mai mare decât i , iar al doilea cel mai mare element este cel mult i .

1.2 Subpunctul 1: Numărătoarea elementelor majore

Pentru subpunctul $c = 1$, observăm că trebuie să numărăm elementele $v[j]$ pentru $j \leq poz$ care satisfac $v[j] \geq v[poz]$. Parcurgând secvența de la început, ne oprim la primul element care nu îndeplinește condiția. Numărul caut este $i - 1$.

Complexitate: $O(poz)$ în cazul cel mai nefavorabil.

1.3 Subpunctul 2: Poziții de maxim record

Pentru subpunctul $c = 2$, ideea centrală este următoarea: parcurgem vectorul de la stânga la dreapta și menținem maximul curent. O poziție i este de interes dacă maximul dintre primele i elemente este exact i . Cu alte cuvinte, elementul de la poziția i trebuie să fie un maxim record, iar valoarea sa trebuie să fie egală cu poziția sa.

```
maxim = 0;
nr = 0;
for (i = 1; i <= n; i++) {
    if (v[i] > maxim)
        maxim = v[i];
    if (maxim == i) {
        nr++;
        if (nr != 1)
            fout << " ";
        fout << i;
    }
}
```

Această abordare necesită o singură parcurgere a vectorului.

Complexitate: $O(n)$

1.4 Subpunctul 3: Poziții încadrate de primele două maxime

Subpunctul $c = 3$ este cea mai complexă variantă. Trebuie să găsim pozițiile i pentru care:

$$\max\{v[1], v[2], \dots, v[i]\} > i \quad \text{și} \quad (1)$$

$$\text{al doilea maxim dintre } v[1..i] \leq i \quad (2)$$

Soluția menține simultan primele două valori maxime întâlnite în parcurgerea secvenței. La fiecare pas i , actualizăm aceste două maxime și verificăm condiția de mai sus.

```
maxim1 = 0;
maxim2 = 0;
nr = 0;
for (i = 1; i <= n; i++) {
    if (v[i] > maxim1) {
        maxim2 = maxim1;
        maxim1 = v[i];
    } else
        if (v[i] > maxim2)
            maxim2 = v[i];
    if (maxim1 > i && maxim2 <= i) {
        nr++;
        if (nr != 1)
            fout << " ";
        fout << i;
    }
}
```

Complexitate: $O(n)$

1.5 Note de implementare

1. Datele de intrare se citesc din fișierul `cartonase.in`, iar rezultatele se scriu în `cartonase.out`.
2. Frecvențele valorilor sunt calculate într-un tablou auxiliar f , deși nu sunt utilizate explicit în soluția finală.
3. La afișare, numerele sunt separate prin spațiu, cu excepția primului număr.
4. Indexarea în cod începe de la 1 (conform convenției uzuale în problemele românești), deci trebuie acordată atenție acestui detaliu la verificarea condițiilor.

1.6 Complexitatea generală

Toate cele trei subpuncte au complexitate liniară în raport cu n , respectiv $O(n)$, și folosesc memorie $O(1)$ în afară de tabloul de intrare. Acest lucru face soluția suficient de eficientă pentru valori mari ale lui n (până la 10^5 , conform dimensiunii tabloului).